Technology Decision Reference

One page per technology. The mechanism that drives the tradeoff, when to reach for it, when it flips, and the line to say in the room.

This sheet answers **"which technology and why"** — the axis a pattern-organized system design sheet doesn't cover. Every page is the same skeleton: **mechanism → reach for / avoid → numbers → CAP → interview line → pushback**. The interview line is the choosing-not-pattern-matching signal; the **pushback** is where the obvious answer reverses — the staff tell.

> **On the numbers:** these are interview **anchors** — order-of-magnitude figures to reason with out loud, not spec-sheet facts. Treat ranges as talking points, not exact values to be tested against.

1 · PostgreSQL

The correct default. Reach for something else only when you can name the specific thing Postgres can't do at your scale.

MECHANISM

B-tree indexes, MVCC, single-leader replication. Postgres does **not** update in place — an UPDATE writes a **new** row version and marks the old dead, so readers snapshot without blocking writers. `VACUUM` reclaims dead versions (hence bloat, HOT updates, index write-amplification). One primary takes writes; replicas stream the WAL, async by default. Write ceiling = local WAL `fsync` + hot-row lock contention, not replicas.

REACH FOR IT WHEN

- You need **multi-row/table ACID transactions** — money, inventory, multi-entity publish.
- Read patterns are **ad-hoc** — planner + indexes handle new WHERE clauses free.
- Joins, aggregations, secondary indexes.
- Scale below "genuinely global + huge." Almost always.

AVOID / AUGMENT WHEN

- >10k sustained writes/s to one logical table SHARD/LSM
- Active-active **multi-region writes** — retrofitted, painful.
- Append-only firehose — LSM fits better.
- **Connection storms** — each conn ≈ a backend process (MBs). Front with **PgBouncer**. Most common real scaling gotcha; hits before raw throughput does.

DURABILITY LAYERS

- **Local WAL fsync** survives a crash, not disk/node loss.
- **Sync replication** (`ANY 1` of several standbys, another AZ) → no acked write on a single disk. Quorum-with-slack, not one named standby (its failure blocks writes).
- **WAL archiving + PITR** survives logical corruption — a bad DELETE replicates faithfully; only PITR saves you.

NUMBERS TO ANCHOR

Point read (indexed)	`< 1 ms`
Writes/s, one primary	`~5k–15k`
Rows before shard hurts	`~10–100M+`
Updates/hot row	`~500–1k/s`
Connections	`pool it`

CAP / CONSISTENCY

CP-leaning — really ACID on one node + tunable replication. Isolation up to SERIALIZABLE is native; async replicas are eventually consistent, `remote_apply` or primary-reads give strong. Transactional ≠ replication consistency.

INTERVIEW LINE

I default to Postgres and make something else justify itself — transactions, joins, query flexibility free, 5–15k writes/s, reads scale on replicas. I move off only when I can name the property it can't provide, and I pool connections before worrying about throughput.

PUSHBACK

"Postgres doesn't scale" is usually wrong — a **hot row**, a **multi-region write**, or **connection exhaustion** flips it, not raw size. Sharded Postgres gets most of Cassandra's write distribution; you lose auto-rebalance and painless geo-replication, not sharding itself.

2 · Cassandra

Query-first, masterless, born multi-region. You trade joins and ad-hoc queries for cheap global replication and painless rebalancing.

MECHANISM

LSM tree + consistent hashing + masterless replication. Writes append to memtable + commitlog, flush to immutable SSTables — sequential I/O, high write ceiling. Compaction merges SSTables. Consistent hashing (vnodes) places partitions on a ring — add a node, token ranges auto-rebalance. **Any node coordinates a write** and forwards to the replica set; no special primary. Consistency tunable per query; multi-DC replication is config.

REACH FOR IT WHEN

- **Known key-based lookups** — "everything for this id," "partition sorted by time."
- Global reads at **local-region latency** — multi-DC out of the box.
- Very high write throughput, append-heavy.
- Capacity by adding nodes, no manual reshard.

THE MODEL TAX (KNOW COLD)

- **No joins, no ad-hoc queries.** Non-partition column = full scan.
- **One table per query** — denormalize, dual-write, own cross-copy consistency.
- No general transactions; LWT (Paxos) is single-partition CAS, slow.
- Incomplete clustering key **silently upserts**.
- **Tombstones**: deletes write markers that linger till compaction; reads across many crater latency. Makes Cassandra a terrible queue. #1 footgun after key upserts.

NUMBERS TO ANCHOR

Write throughput	`very high, scales out`
Single-partition read	`low ms`
Add capacity	`join a node`
R + W > N	`→ strong reads`

CAP / CONSISTENCY

AP by design, **tunable toward CP**. QUORUM reads + QUORUM writes → R+W>N → read overlaps latest write → strong, at latency/availability cost. Even at ALL you get **replication** consistency, never **transactional**.

INTERVIEW LINE

Cassandra fits keyed lookups read globally, written rarely. Its edge over sharded Postgres isn't write throughput at my volume — it's automatic rebalancing and multi-region replication as a config line, not a project. I accept no joins and eventual consistency (tunable to strong via QUORUM/QUORUM).

PUSHBACK

Below genuine global scale, **Postgres is the better default** even for keyed data — you keep transactions and query flexibility and defer the operational tax (repair, compaction, tombstone tuning). Right only when scale × geo × access-simplicity all hold. Never model a queue/high-churn-delete on it — tombstones punish you.

3 · Redis

In-memory, microsecond ops. A Swiss-army knife: cache, counter, lock, rate limiter, ephemeral queue, leaderboard. Rarely your source of truth.

MECHANISM

RAM data structures, single-threaded command execution. Every command (and Lua script) runs atomically, no locks — the load-bearing property. (Modern Redis uses I/O threads for network, but command execution stays serialized, so atomicity holds.) Durability optional: `RDB` snapshots + `AOF` (`everysec`). Replicas + Sentinel/Cluster for HA. A fast cache that **can** persist, not a DB that's fast.

REACH FOR IT WHEN

- **Cache** in front of a slower store (default use).
- **Atomic counters** — INCR/DECR: rate limits, admission, fan-in.
- **Distributed lock** (SET NX PX + fencing token).
- **ZSET** — leaderboards, waiting-room order, timer queues.
- Ephemeral state with **TTL auto-expiry**.

CACHE PATTERNS TO NAME

- **Cache-aside** (lazy): check cache, on miss read DB + populate. Default. Stale risk → TTL / invalidate on write.
- **Write-through/behind**: write hits cache (+ DB). Fresher, more coupled.
- **Stampede**: hot key expires, thousands hit DB to refill. Fix: lock the refill (one repopulates), coalesce, or probabilistic early expiry.

AVOID / CAREFUL WHEN

- It's your **only** copy — failover to a lagging replica loses writes.
- Dataset > RAM — memory-bound is the cost model.
- TTL as a business trigger: expiry **deletes a key**, doesn't run compensation. Notifications are at-most-once.

NUMBERS

Op latency	`~0.1–0.5 ms`
Throughput/node	`~100k+ ops/s`
Bound	`RAM + network`

CAP / CONSISTENCY

Atomic per-key ops, but durability/HA is weak by default — async replication + Sentinel failover can lose acked writes. Cluster shards by key; multi-key atomicity only within a hash slot.

INTERVIEW LINE

Redis when I need one thing very fast and can tolerate losing it: cache, atomic counter, TTL'd hold. Serialized execution makes every op and Lua script atomic — right for admission control and rate limiting. Durable truth stays in Postgres behind it, and I plan for stampede on hot keys.

PUSHBACK

For hold/expiry, Redis TTL only wins if the **key IS the hold**. If Postgres owns the counter, a DB sweeper beats notifications — durable, self-healing. Redis for ticket-onsale scale (100k holds/s, visible countdown); sweeper for hotel scale.

4 · Kafka

A durable, ordered, replayable log. Not a queue you drain — a commit log consumers read at their own offset. The backbone for decoupling and buffering.

MECHANISM

Partitioned append-only log; consumers track offsets. Order guaranteed **within** a partition, not across. Messages persist for a retention window — replay by rewinding offset. Producers key messages (same key → same partition → ordered). Delivery **at-least-once** by default; consumers must be idempotent.

REACH FOR IT WHEN

- **Decouple** producers from consumers.
- **Buffer a firehose** — absorb spikes, load-level.
- **Fan-out** one event to many consumer groups.
- **Event sourcing / CDC** — log is truth, replay to rebuild.

OPERATIONAL REALITIES

- **Partition count = parallelism ceiling**: consumers ≤ partitions (extras idle). Awkward to reduce — size for peak.
- **Rebalancing**: consumer join/leave reassigns partitions, pausing processing. Flapping = classic incident.
- **Order vs parallelism**: order within partition; key by the entity whose order matters.

AVOID / CAREFUL WHEN

- Need **per-message ack/delete**, priority, delay — that's a queue.
- Need **global ordering** — per-partition only.
- Low-volume job dispatch — operational weight you may not need.

NUMBERS

Throughput	`very high`
Ordering	`per-partition`
Delivery	`≥ once`
Parallelism	`≤ partitions`

CAP / CONSISTENCY

CP-leaning: partition leader + ISR; `acks=all` waits for ISR (latency for no-loss). Exactly-once (EOS) is **within Kafka only** — not for external side effects (your DB/API), which still need idempotency.

INTERVIEW LINE

Kafka to decouple and buffer: consumer groups read independently at their offset, retention lets me replay to rebuild or add a consumer. Key by entity for per-partition order, size partitions for peak parallelism, make consumers idempotent — EOS doesn't cover my DB write.

PUSHBACK

Kafka on a simple job queue is over-engineering — for per-message ack, retry, priority, DLQ without replay/fan-out, a **task queue is simpler**. Kafka earns its cost with replay, multiple consumers, or firehose buffering — name which.

5 · Message Queue (SQS / RabbitMQ)

Per-message work dispatch with ack, redelivery, and dead-lettering. The "distribute tasks to workers" primitive — distinct from Kafka's replayable log.

MECHANISM

A broker holds messages; workers pull one, process, and **ack** to delete it. No ack (crash/timeout) → message becomes visible again and redelivers. After N fails → **dead-letter queue** for inspection. Unlike a log, a consumed+acked message is **gone** — no offset to rewind, no replay, no second consumer group re-reading it.

REACH FOR IT WHEN

- **Task/job dispatch** to a worker pool — resize, thumbnails, webhooks.
- Want **per-message retry + DLQ** without building it.
- **Priority** or **delayed/scheduled** delivery.
- Decoupling with **no** replay/fan-out — competing consumers drain one queue.

LOG VS. QUEUE — SAY THIS

- **Kafka (log)**: retained, replayable, many consumer groups, per-partition order. "Many readers, rewindable history."
- **Queue**: consumed-and-deleted, per-message ack/retry/DLQ, competing consumers. "One logical reader, work drains."
- Rule: replay or multiple consumers of the same event → **log**. Work distribution with retry/DLQ → **queue**.

AVOID / CAREFUL WHEN

- Need to **replay** or add a consumer seeing past events — that's a log.
- **Strict global ordering** — mostly not guaranteed (SQS FIFO trades throughput).
- High fan-out to many independent subscribers — pub/sub or log.

NUMBERS

Delivery	≥ once
Ordering	best-effort (FIFO opt-in)
Retry	built-in
DLQ	built-in
Replay	✗

CAP / CONSISTENCY

Managed queues favor availability + at-least-once; ordering/dedup opt-in (FIFO) at throughput cost. Make consumers **idempotent** regardless — redelivery is normal, not exceptional.

INTERVIEW LINE

For distributing background work with automatic retry and dead-lettering, a task queue, not Kafka. Workers ack on success; a crash redelivers after the visibility timeout, poison messages fall to a DLQ. Kafka only when I need replay or multiple independent consumers of the same stream.

PUSHBACK

The mistake is defaulting to Kafka for everything "async." One logical consumer draining work with retry/DLQ for free → a queue is simpler and cheaper. Flip to a log the moment you need replay, event fan-out, or an audit trail.

6 · S3 / Blob Storage

Effectively infinite, cheap, durable bytes over HTTP. The rule: large binaries live here, your DB stores the pointer, and the bytes never transit your API tier.

MECHANISM

Key→object store over HTTP, ~11-nines durability. Not a filesystem — flat keyspace, immutable blobs (overwrite, don't edit). Standard classes replicate across ≥3 AZs (One Zone classes deliberately don't — cheaper, less durable). Tiered storage trades retrieval latency for cost. **Presigned URLs** let clients transfer directly with time-limited creds. **Multipart** splits large objects into parallel, resumable parts.

REACH FOR IT WHEN

- Any **large binary**: video, images, uploads, backups, ML artifacts, logs.
- Static assets to serve via CDN (S3 as origin).
- Data-lake / event archive (query later via Athena).
- Decoupling upload transfer from app servers.

TWO PATTERNS TO SAY

- **Presigned direct upload** — client PUTs to S3; API only issues URLs. Bytes bypass your tier.
- **Multipart** — one URL per part, parallel, per-part retry, resumable via ListParts. 5 MB min / 10k parts max.

NUMBERS TO ANCHOR

Durability	`~11 nines (multi-AZ)`
Single PUT cap	`5 GB`
Part min / max	`5 MB / 10k`
First-byte latency	`tens of ms`

CAP / CONSISTENCY

Strongly read-after-write consistent for all ops since Dec 2020 — a GET after a PUT returns the latest. No cross-object transactions, no conditional multi-key atomicity, no querying inside objects. A durable key→blob store, not a database.

INTERVIEW LINE

Large binaries go to blob storage, the DB keeps only the key. Clients upload via presigned URLs so gigabytes never touch my API tier; big files use multipart — parallel, resumable. Front with a CDN for reads, and drive state transitions off object-created events rather than trusting the client's "done."

PUSHBACK

S3 is now strongly consistent — don't cite the old eventual-consistency caveat; the real limits are no transactions and no querying inside objects. And it's **pointer-in-DB**: an object with no DB row is an orphan you pay for — reconcile with a sweeper.

7 · CDN

Cache bytes at edge PoPs close to users. The answer to "global users" and "read-heavy static/cacheable content." Turns 20k QPS for one object into ~one origin fetch per PoP.

MECHANISM

Geographically distributed edge caches. User hits nearest PoP; cache hit serves locally (low latency, zero origin load). Miss fetches from origin, caches per TTL/headers. `Cache-Control` / `ETag` govern freshness. Collapses fan-out: N users on one object = ~1 origin fetch per PoP per TTL. Absorbs spikes and DDoS at the edge.

REACH FOR IT WHEN

- **Static assets**: JS/CSS/images, video segments, downloads.
- **Global read audience** for the same content.
- A **hot cacheable value** read enormously — waiting-room "admitted" counter cached 2s collapses 500k polls to ~0 origin.
- Shielding origin from spikes.

AVOID / CAREFUL WHEN

- **Personalized / per-user** responses — low hit rate (edge compute is the escape hatch).
- Highly dynamic data where staleness is unacceptable.
- Invalidation is hard — **short TTL** beats active purge when seconds of staleness are OK.

NUMBERS

Edge hit latency	`single→tens ms`
Origin offload	`often > 95%`
Invalidation	`TTL > purge`

CAP / CONSISTENCY

Deliberately eventually consistent — the edge serves possibly-stale content bounded by TTL. That staleness **is** the tradeoff for latency and offload; design around "what's cacheable, for how long," not "how do I make the edge fresh."

INTERVIEW LINE

A CDN pushes cacheable bytes to edge PoPs so global reads hit a nearby cache — that's how one popular object survives 20k QPS at ~one origin fetch per PoP. I lean on short TTLs over active invalidation whenever seconds of staleness are acceptable, which is most static content.

PUSHBACK

A CDN does nothing for **personalized or write** paths — don't wave it at a dynamic problem. The real question is always **invalidation**: what's cacheable and for how long. Staleness tolerance is the actual variable.

8 · Vector Database

Approximate nearest-neighbor search over embedding vectors. The retrieval half of RAG: "find the k chunks most semantically similar to this query."

MECHANISM

ANN index over high-dim vectors. Text/images → embeddings (~768–3072 dims). Query embedded the same way; DB finds neighbors by cosine/dot distance. Exact search is O(N)/query, so it uses an **approximate** index — usually `HNSW` (navigable small-world graph): log-ish search, tunable recall↔latency. Metadata filtering (tenant, date, ACL) runs alongside — critical, easy to get wrong.

REACH FOR IT WHEN

- **RAG**: ground an LLM in your corpus — top-k chunks into context.
- Semantic search (meaning, not keywords).
- Recommendations / dedup / similarity.

PIPELINE TO SAY

Ingest: chunk → embed → upsert w/ metadata. Query: embed → ANN top-k → filter by ACL → rerank → prompt. **Chunking + reranking** move quality more than the DB.

AVOID / CAREFUL WHEN

- Keyword/exact-match wanted — **BM25/Elasticsearch** may beat or hybridize.
- Small corpus — FAISS or `pgvector` beats a new system.
- **Permission leakage**: embeddings ignore ACLs. Filter by permission at query time or retrieve across tenants.

NUMBERS

Embedding dims	`~768–3072`
Raw vector	`dims × 4 B (f32)`
Quantized	`int8 ~4×, bin ~32×`
1M×1536	`~6 GB + HNSW graph`

CAP / CONSISTENCY

Usually eventually consistent, read-optimized — a just-upserted vector may not be immediately searchable (fine for RAG). The real gotcha is **memory**: the HNSW graph often exceeds the raw vectors; quantization is how you fit large corpora in RAM.

INTERVIEW LINE

For RAG I index chunks with HNSW, then embed the question, pull top-k by cosine, filter by the user's ACL, rerank before the prompt. Chunking and reranking drive quality more than which vector DB. Below a few million vectors, pgvector; above, budget for index memory + quantization.

PUSHBACK

Vector search isn't always the answer — **hybrid** (BM25 + vector) often wins; pure semantic misses exact terms (names, IDs, codes). The sharp failure mode is **permissions**: sensitive data in embeddings/logs, cross-tenant retrieval. Raise ACL-at-query-time unprompted.

9 · Workflow Orchestrator

Temporal / Step Functions. Durable multi-step workflows with fan-out/fan-in and exactly-once continuation. The answer to "coordinate N async tasks and know when all are done."

MECHANISM

Event-sourced state machine + replay. Every transition appends to a durable history log — that log is truth, not in-memory state. On crash, the workflow re-executes from the top, but completed steps return **recorded results** from the log; real execution resumes only past the end of history. Fan-in is a read over the ordered log, decided by a **single evaluator** — no concurrent-counter race. Requires **deterministic** workflow code (no clocks/random outside activities).

REACH FOR IT WHEN

- **Fan-out/fan-in**: spawn N, run one step when all finish (transcode → READY).
- Long-running, multi-step, must survive crashes (saga, provisioning).
- Steps need **retries, timeouts, compensation** without hand-rolled state.
- You'd otherwise build a status table + counter + reaper.

AVOID / CAREFUL WHEN

- Single-step / fire-and-forget — a queue is enough.
- Ultra-low-latency sync paths (replay + persistence overhead).
- **Activities are at-least-once** — continuation is exactly-once, but activity code must be idempotent.

TWO FLAVORS

Temporal	`your code + replay`
Step Functions	`declarative JSON FSM`
Fan-in	`built-in primitive`
Guarantee	`exactly-once continuation`

CAP / CONSISTENCY

Durability comes from its backing store (Temporal: Cassandra/ Postgres/MySQL; SF: managed). It gives exactly-once **continuation** layered on at-least-once **activity execution** — different things; activities still need idempotency.

INTERVIEW LINE

To fan out N tasks and fire one step when all complete, an orchestrator instead of a hand-rolled counter-and-reaper. It persists every transition to a durable log and decides completion with a single reader over that ordered log, so the fan-in race is gone by construction. Continuation is exactly-once; I still make activities idempotent.

PUSHBACK

Not for a single async job — that's a queue, and replay/determinism is real cost. The mechanism to **name** even if you hand-roll: atomic `DECR` for "am I last," a guarded status transition for idempotent completion, a reaper for stalls.

10 · Elasticsearch

An inverted index for full-text and faceted search. A secondary read model you feed from your primary store — not your source of truth.

MECHANISM

Inverted index (Lucene) + distributed shards. Text is analyzed (tokenized, lowercased, stemmed) into terms; the index maps term → documents, so full-text queries are fast. Relevance ranked by `BM25`. Shards + replicas for scale/HA. **Near-real-time** — a refresh interval (~1s) gates visibility. Async-fed from your primary via CDC or dual-write.

REACH FOR IT WHEN

- **Full-text search** with relevance ranking.
- **Faceted / filtered** search — aggregations across many fields.
- Log / observability search at scale (ELK).
- Hybrid with vectors for semantic + keyword.

AVOID / CAREFUL WHEN

- As a **primary store** — a derived index; rebuild from truth.
- Need **strong consistency / transactions** — NRT, no ACID.
- Simple exact lookups — a DB index is cheaper.
- Sync is the real cost — CDC pipeline + reconciliation.

NUMBERS

Visibility	`NRT (~1s)`
Ranking	`BM25`
Role	`derived read model`

CAP / CONSISTENCY

AP-leaning, eventually consistent with your primary by construction (async-fed, ~1s refresh). Treat divergence as normal: design the sync pipeline, a reconciliation path, and the ability to rebuild from source at any time.

INTERVIEW LINE

Full-text and faceted search go to Elasticsearch as a secondary read model fed via CDC — the inverted index and BM25 give ranking a B-tree can't. It's NRT and not my source of truth, so I design the sync pipeline and reconciliation, and can rebuild the index from the primary anytime.

PUSHBACK

The trap is treating it as a database. It's a **derived index** — the hard part is the sync pipeline and its consistency, not the query. For semantic search, **hybrid BM25 + vector** usually beats either. For exact-match, a Postgres index is simpler and consistent.

11 · Decision Matrix

Collapse the ten stores into one glance. Match the workload to the default pick; the "because" is the one-line justification.

WORKLOAD	DEFAULT PICK	BECAUSE
Transactions, joins, ad-hoc queries	**PostgreSQL**	ACID + query flexibility; the default until something breaks
Keyed lookups, global reads, huge scale	**Cassandra**	auto-rebalance + multi-region writes; you don't need joins
Cache / counter / lock / TTL'd state	**Redis**	in-memory, atomic ops, microsecond latency
Decouple + replay + multiple consumers	**Kafka**	retained ordered log, rewindable, fan-out
Background jobs with retry + DLQ	**Queue (SQS/RabbitMQ)**	per-message ack/redelivery; no replay needed
Large binaries (video, images, uploads)	**S3 / Blob**	infinite cheap durable bytes; keep the pointer in the DB
Global static / hot read-heavy content	**CDN**	serve from the edge; collapse read fan-out
Semantic search / RAG retrieval	**Vector DB**	ANN over embeddings; hybrid with BM25 for exact terms
Coordinate N tasks, fan-in, sagas	**Orchestrator**	durable fan-out/fan-in, exactly-once continuation
Full-text / faceted search	**Elasticsearch**	inverted index + BM25; a derived read model, not truth

THE META-RULE

Most systems are **Postgres + one or two specialists**: Postgres as the transactional source of truth, plus a cache (Redis), a blob store (S3), a CDN, and **maybe** a search or vector index fed from Postgres via CDC. Reach for Cassandra / Kafka / an orchestrator only when a named requirement — global write scale, replay/fan-out, durable multi-step coordination — forces it. Naming the **one** store as truth and treating the rest as derived is the coherence signal.

CROSS-CUTTING PRINCIPLES THAT REAPPEAR EVERYWHERE

- **Idempotency** is non-negotiable anywhere delivery is at-least-once (Kafka, queues, orchestrator activities, retried writes).
- **One source of truth**; everything else (cache, search index, vector store, denormalized copy) is derived and must be rebuildable + reconciled.
- **Reads tolerate staleness; writes must be correct** — the asymmetry behind CDNs, replicas, caches, and search indexes.
- **Move a guarantee out of the database only when arrival rate makes the DB physically unable to provide it** — the same rule for sharding, Redis holds, and counter offloading.